

Compressió de dades i imatges - GEI - FIB

Tema 3: Codificació aritmètica

Anna de Mier, Lluís Vena
anna.de.mier@upc.edu, lluis.vena@upc.edu

Departament de Matemàtiques • Universitat Politècnica de Catalunya

Març 2026

Capítol 4 de [Introduction to Data Compression](#) Sayood, Khalid Morgan
Kaufmann, 2012, 4th ed.

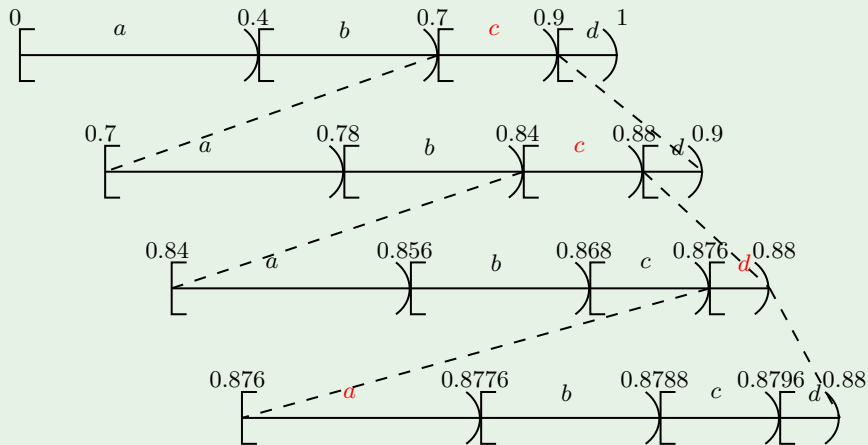
Presentació

- Hem vist que fent servir Huffman amb extensions de la font^a ens podem acostar tant com vulguem a l'entropia
- Però això normalment no és ni pràctic ni eficient
- Per exemple, considerem la font $\{(a, 0.4), (b, 0.3), (c, 0.2), (d, 0.1)\}$ i volem codificar **ccda**. Per fer una 4-extensió de la font, hauríem de calcular les 4^4 paraules del codi associat i triar la corresponent a **ccda**.
- La forma ideal seria poder trobar la paraula del codi sense necessitat de construir tot el codi.
- La *codificació aritmètica* aconsegueix això associant a cada missatge *una única paraula binària*
- Més concretament, a cada missatge se li associa un interval $[m, M) \subseteq [0, 1)$. Després s'escull un nombre real en aquest interval i es retorna la seva representació binària, convenientment truncada si cal.^b

^aés a dir, empaquetant símbols de 2 en 2, de 3 en 3, etc

^bPer exemple, $0100110 = 0\frac{1}{2^1} + 1\frac{1}{2^2} + 0\frac{1}{2^3} + 0\frac{1}{2^4} + 1\frac{1}{2^5} + 1\frac{1}{2^6} + 0\frac{1}{2^7} = \frac{19}{64}$.

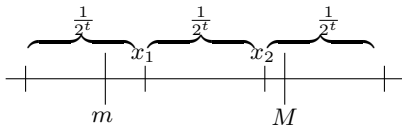
Exemple: $(a, 0.4), (b, 0.3), (c, 0.2), (d, 0.1), \text{ccda}$



El missatge estarà representat per un real de l'interval $[0.876, 0.8776) \equiv [m, M)$.

Qualsevol valor de l'interval $[m, M)$ serveix per representar el missatge!

Opció 1: sigui $t \in \mathbb{N}$ tal que $\frac{1}{2^t} \leq M - m < \frac{1}{2^{t-1}}$. Sigui $x \in \mathbb{N}$ tal que $m \leq \frac{x}{2^t} < M$ o equivalentment $m2^t \leq x < M2^t$; si hi ha dos valors possibles, es tria el valor parell.



El missatge es representa per $\frac{x}{2^t}$.

Exemple anterior [0.876, 0.8776)

$\frac{1}{2^t} \leq 0.8776 - 0.876 = 0.0016$, $t = 10$, $2^t = 1024$. $897.024 \leq x < 898.662$. El valor seria

$$\begin{aligned} \frac{898}{1024} &= \frac{449}{512} = \frac{256 + 128 + 64 + 0 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 0 \cdot 2 + 1}{512} \\ &= \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{0}{16} + \frac{0}{32} + \frac{0}{64} + \frac{0}{128} + \frac{0}{256} + \frac{1}{512} = (0.111000001)_2 \end{aligned}$$

El codi seria 111000001 (449 en binari amb 9 bits $512 = 2^9$). Si haguéssim tingut $x = 7$, el codi seria 0000000111 (10 bits $1024 = 2^{10}$).

Opció 2: Trobar l'expressió binària de m i M :

$$m = 0, a_1 a_2 \dots a_r 0 \dots$$

$$M = 0, a_1 a_2 \dots a_r 1 \dots$$

↑ primer bit en què difereixen

Retornem $a_1 a_2 \dots a_r 1$ com a codificació excepte si:

❗ si $M = 0.a_1 a_2 \dots a_r 1$:

- ▶ si l'expressió de m és finita retornem m ,
- ▶ si no ho és, s'envia $a_1 a_2 \dots a_r 0 a_{r+2} \dots a_s 1$ essent $a_{r+2} = \dots = a_s = 1$, $a_{s+1} = 0$ en la representació de m^* ; és a dir, s'envien els 1's que venen (que podrien ser zero) i el 0 posterior es transforma en 1.

Exemple

$$m = 0.10101|0|11001\dots$$

$$M = 0.10101|1| \quad \updownarrow$$

s'envia 10101 0 11 **1**

*pels nombres que acaben en 1 periòdic prenem sempre la representació finita

Tornant a l'exemple anterior [0.876, 0.8776)

$$m = 0.876 = 0.11100000|0|1000$$

$$M = 0.8776 = 0.11100000|1|0101$$

El codi que s'envia és 111000001, 9 bits. (Notem que $\log \frac{1}{p} = \log \frac{1}{0.2 \cdot 0.2 \cdot 0.1 \cdot 0.4} \approx 9.3$ bits.)

Opció 3: prenem el punt mig de l'interval $(M + m)/2$, l'escrivim en binari i trunquem a $\lceil \log \frac{1}{M-m} \rceil + 1$ bits

En aquest cas, el resultat és un codi prefix

De nou $[0.876, 0.8776)$

$$\frac{m + M}{2} = 0.8768 = 0.1110000001110 \dots$$

El codi que s'envia és $11100000011=0.8764\dots$, 11 bits.

Observacions:

- Amb la primera i la tercera opcions cal esperar a tenir tot el missatge per començar a codificar-lo.
- Amb la segona es pot començar a codificar tan bon punt hi hagi bits que coincideixin en la representació binària de m i M .

- **Recordatori de PE:** Donats $s_1, \dots, s_n$ amb probabilitats $p_1 \geq p_2 \geq \dots \geq p_n > 0$ definim la funció de distribució[†]

$$F(i) = \sum_{k=1}^i p_k$$

En el cas de l'exemple 3 si $a \leftrightarrow 1$, $b \leftrightarrow 2$, $c \leftrightarrow 3$, $d \leftrightarrow 4$

$$F(0) = 0, \quad F(1) = 0.4, \quad F(2) = 0.7, \quad F(3) = 0.9, \quad F(4) = 1.$$

[†]També anomenada funció de probabilitat acumulada

Algorisme

- **Codificar:** Donada la seqüència $s_{i_1} s_{i_2} s_{i_3} \dots s_{i_r}$

$$m^{(0)} = 0 \quad M^{(0)} = 1$$

Per a $k = 1, \dots, r$

$$d_{k-1} = M^{(k-1)} - m^{(k-1)}$$

$$m^{(k)} = m^{(k-1)} + F(i_k - 1)d_{k-1}$$

$$M^{(k)} = m^{(k-1)} + F(i_k)d_{k-1}$$

Exemple: *ccdabb*

- $[0, 1)$
- $c \mapsto [0.7, 0.9)$
- $c \mapsto [0.84, 0.88)$
- $d \mapsto [0.876, 0.88)$
- $a \mapsto [0.876, 0.8776)$
- $b \mapsto [0.87664, 0.87712)$
- $b \mapsto [0.876832, 0.876976)$

Algorisme

- **Descodificar:** Donat x_1 i la longitud del missatge. Per a $i = 1, \dots$, longitud del missatge:
 - ▶ Si $F(k-1) \leq x_i < F(k)$ retornar el símbol s_k .
 - ▶ $x_{i+1} = \frac{x_i - F(k-1)}{F(k) - F(k-1)}$.

Exemple: Si hem rebut 111000001

- $x_1 = 0.876953125 \mapsto c$
- $x_2 = 0.884765625 \mapsto c$
- $x_3 = 0.923828125 \mapsto d$
- $x_4 = 0.23828125 \mapsto a$
- $x_5 = 0.595703125 \mapsto b$
- $x_6 = 0.65234375 \mapsto b$
- ...

Podríem seguir indefinidament, per això és necessari donar la longitud del missatge o incloure un símbol EOF.

Observacions:

- i) És necessari donar la longitud del missatge o incloure un símbol EOF^a.
- ii) Si hem codificat la seqüència $s_1 \dots s_n$, l'interval $[m, M)$ té longitud $p_{i_1} \dots p_{i_k}$
- iii) El número que es retorna en codificar es pot construir com es vulgui, l'important és que estigui confinat en l'interval semiobert $[m, M)$. Si volem aconseguir un codi prefix, sempre podem trobar un nombre de l'interval $[m, M)$ que escrit en binari tingui longitud com a molt $\lceil -\log(M - m) \rceil + 1$ bits (és a dir, un bit més de la informació de $s_1 \dots s_n$)

^aCaldria assignar a aquest símbol una probabilitat molt petita

La codificació aritmètica pot aconseguir un codi prefix amb longitud mitjana menor que $nH(\mathcal{S}) + 2$ (on n és la longitud del missatge).

Proof.

Si l'interval $[m, M)$ té longitud $p_{i_1} \cdots p_{i_n}$, l'Opció 3 ens dona menys de $\log \frac{1}{p_{i_1} p_{i_2} \cdots p_{i_n}} + 2$ bits Aleshores

$$\begin{aligned}\bar{l} &= \sum p_{i_1} \cdots p_{i_n} l_{i_1 \dots i_n} < \sum p_{i_1} \cdots p_{i_n} \left[\log \frac{1}{p_{i_1} \cdots p_{i_n}} + 2 \right] \\ &= nH(\mathcal{S}) + 2\end{aligned}$$

on la darrera igualtat surt de manipulacions anàlogues a les de la demostració del teorema d'extensió de la font.



Necessitem precisió infinita!

Possibles solucions, no excloents:

- Anar reescalant l'interval a mesura que tenim “bits segurs”
- Treballar amb enters en un interval $[0, R) \subseteq \mathbb{Z}$ en comptes de reals de $[0, 1)$ (codificació aritmètica entera). En aquest cas, en comptes de probabilitats, treballem amb freqüències $f_i \in \mathbb{N}$.

Reescalat (I)

Considerem els intervals:

$$I_1 = [0, 0.5), \quad I_2 = [0.5, 1), \quad I_3 = [0.25, 0.75)$$

- Si $[m, M) \subseteq I_1$, el primer bit serà un 0 i reescalem $E_1(x) : [0, \frac{1}{2}) \rightarrow [0, 1)$, $E_1(x) = 2x$.
- Si $[m, M) \subseteq I_2$, el primer bit serà un 1 i reescalem $E_2(x) : [\frac{1}{2}, 1) \rightarrow [0, 1)$, $E_2(x) = 2x - 1$.
- Si $[m, M) \subseteq I_3$, apliquem $E_3(x) : [\frac{1}{4}, \frac{3}{4}) \rightarrow [0, 1)$, $E_3(x) = 2x - \frac{1}{2}$ i esperem (*).

(*) Si cal esperar N vegades i després es cau en:

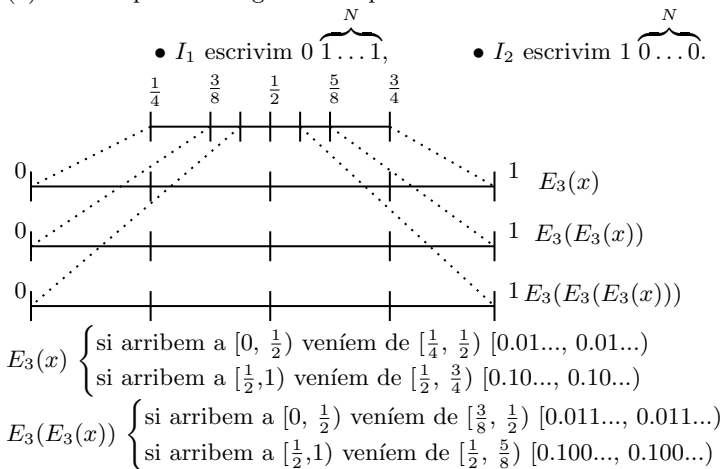
- I_1 escrivim $0 \overbrace{1 \dots 1}^N$,

- I_2 escrivim $1 \overbrace{0 \dots 0}^N$.

Quan no es pot aplicar cap més reescalat, passem a processar el símbol següent del missatge.

Reescalat (II)

(*) Si cal esperar N vegades i després es cau en:



Treballar amb enters $[0, R)$

Combinem el reescalat anterior amb treballar amb nombres enters

Si els símbols apareixen amb freqüència $f_i \in \mathbb{N}$, sigui $T = \sum_i f_i$.

Definim R tal que $R > 4T$ (perquè l'interval $[m, M)$ pot arribar a ser com a màxim $\frac{1}{4}$ de l'interval total abans de ser reescalat; en aquest quart de l'interval hi ha de cabre T per poder discernir entre els diferents símbols).

(Més detalls al laboratori i a Ateneu)